WEEK-4 PRACTICAL

Task1:

Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()` . Compare the behavior of both functions.



```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define NUM_CHILDREN 3

int main() {
    pid_t child[NUM_CHILDREN];
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n\n", getpid());

    // Create 3 child processes
    for (int i = 0; i < NUM_CHILDREN; i++) {

        child[i] = fork();

        if (child[i] < 0) {
            perror("fork failed");
            exit(1);
        }

        if (child[i] == 0) {
            // Child process
            printf("Child %d started. PID = %d, Parent PID = %d\n",
                   i + 1, getpid(), getppid());

            // Different execution times
            sleep((i + 1) * 2);

            printf("Child %d completed. PID = %d\n", i + 1, getpid());

            exit(10 + i);
        }
    }

    printf("\nAll 3 child processes have been created.\n");

    // Demonstrate wait()
    printf("\nUsing wait():\n");

    pid_t finished = wait(&status);

    if (finished > 0) {
        if (WIFEXITED(status)) {
            printf("wait() detected child PID %d finished with exit status %d\n",
                   finished, WEXITSTATUS(status));
        }
    }

    // Demonstrate waitpid()
    printf("\nUsing waitpid():\n");

    for (int i = 0; i < NUM_CHILDREN; i++) {

        // Skip the child already collected by wait()
        if (child[i] == finished)
```

```
^G Get Help   ^O WriteOut   ^R Read File   ^Y Prev Pg   ^K Cut Text   ^C Cur Pos
^X Exit       ^J Justify    ^W Where is    ^V Next Pg   ^U UnCut Text ^T To Spell
```




```
Last login: Mon Aug 17 19:06:30 on ttys000
(eval):8: command not found: 0
[manikanta@Hemu] ~ % cd ~/Desktop/os
[manikanta@Hemu] ~/Desktop/os % mkdir week4pract
[manikanta@Hemu] ~/Desktop/os % cd week4pract
[manikanta@Hemu] week4pract % nano task1.c
[manikanta@Hemu] week4pract % gcc task1.c -o task1
[manikanta@Hemu] week4pract % ./task1
Parent Process Started
Parent PID: 7750

Child 1 started. PID = 7754, Parent PID = 7750
Child 2 started. PID = 7755, Parent PID = 7750

All 3 child processes have been created.

Using wait():
Child 3 started. PID = 7756, Parent PID = 7750
Child 1 completed. PID = 7754
wait() detected child PID 7754 finished with exit status 10

Using waitpid():
Child 2 completed. PID = 7755
waitpid() detected child PID 7755 finished with exit status 11
Child 3 completed. PID = 7756
waitpid() detected child PID 7756 finished with exit status 12

Parent process completed.
manikanta@Hemu week4pract %
```

Task2:

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

2a)Create a zombie:

```
  UW PICO 5.09                                File: task2_zombie.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        // Child
        printf("Child process: PID = %d\n", getpid());
        printf("Child process terminating...\n");

        exit(0);
    }
    else {
        // Parent
        printf("Parent process: PID = %d\n", getpid());
        printf("Child process: PID = %d\n", pid);

        printf("Parent sleeping...\n");

        // Parent does NOT call wait()
        sleep(30);

        printf("Parent terminating...\n");
    }

    return 0;
}

^G Get Help   ^O WriteOut   ^R Read File   ^Y Prev Pg   ^K Cut Text   ^C Cur Pos
^X Exit       ^J Justify    ^W Where is    ^V Next Pg   ^U UnCut Text ^T To Spell
```

```
[manikanta@Hemu week4pract % nano task2_zombie.c
[manikanta@Hemu week4pract % gcc task2_zombie.c -o task2_zombie
[manikanta@Hemu week4pract % ./task2_zombie
Parent process: PID = 7813
Child process: PID = 7814
Parat sleeping...
Child process: PID = 7814
Child process terminating...
Parent terminating...
[manikanta@Hemu week4pract %
```

```
[manikanta@Hemu week4pract % gcc task2_zombie.c -o task2_zombie
[manikanta@Hemu week4pract % ./task2_zombie
 Parent process: PID = 7813        Child process: PID = 7814        Child process terminating...
Parent terminating...
[manikanta@Hemu week4pract %
```

2b)Eliminate the zombie:




```
[manikanta@Hemu week4pract % nano task2_fixed.c
[manikanta@Hemu week4pract % gcc task2_fixed.c -o task2_fixed
[manikanta@Hemu week4pract % ./task2_fixed
Parent process: PID = 7840
Child process: PID = 7841
Parent waiting for child...
Child process: PID = 7841
Child process terminating...
Child exited normally with status = 0
Parent terminating...
manikanta@Hemu week4pract %
```